POLITECHNIKA GDAŃSKA

WYDZIAŁ ELEKTRONIKI, TELEKOMUNIKACJI I INFORMATYKI

Imię i nazwisko studenta: Jakub Ławicki
Nr albumu: 189788
Poziom kształcenia: studia drugiego stopnia
Forma studiów: stacjonarne
Kierunek studiów: Elektronika i telekomunikacja
Specjalność: Systemy wbudowane czasu rzeczywistego

PRACA DYPLOMOWA MAGISTERSKA

Tytuł pracy w języku polskim: Badanie wpływu różnych metod optymalizacji kodu na wydajność systemu wbudowanego opartego na systemie Linux.

Tytuł pracy w języku angielskim: Investigation of the impact of various code optimization methods on the performance of a Linux-based embedded system.

Opiekun pracy: dr hab. inż. Jacek Marszal

STRESZCZENIE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Słowa kluczowe: Optymalizacja algorytmów, systemy wbudowane, system operacyjny Linux

Dziedzina nauki i techniki zgodna z OECD Nauki inżynieryjne i techniczne, Elektrotechnika, elektronika i inżynieria informatyczna,

ABSTRACT

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Keywords: Self-organizing maps, unsupervised learning, clustering, classification, WTM algorithm

OECD consistent field of science and technology classification: Engineering and technology, Electrical engineering, Electronic engineering, Information engineering,

SPIS TREŚCI

Spis treści **5**

1 Wstęp i cel pracy **8**
1.1 Cel pracy 8
1.2 Zawartość rozdziałów 8

2 Architektura systemów operacyjnych w zastosowaniach wbudowanych **9**
2.1 Rola systemów operacyjnych w systemach wbudowanych 9
2.1.1 Model warstwowy systemów operacyjnych 9
2.1.2 Systemy wbudowane czasu rzeczywistego wraz z ich podziałem 11
2.1.3 Z czego wynika koniecznosc stosowania systemow operacyjnych a nie bare metal 11
2.2 Budowa i architektura systemu Linux 11
2.2.1 Jądro systemu 11
2.2.2 Mechanizm stronnicowania pamięci i wykorzystanie pamieci cache 11
2.2.3 wykorzystanie systemu Linux w systemach wbudowanych 11
2.3 Procesy i wątki w systemie Linux 11
2.4 Szeregowani zadań i determinizm czasowy w systemach wbudowanych opartych na Linuxie 11

3 Wybrany algorytm przetwarzania danych i techniki optymalizacji kodu **12**
3.1 Teorytyczne podstawy analizy algorytmów 12
3.2 Opis i analiza Szybkiej Transformaty Fouriera 12
3.3 Warianty algorytmu Szybkiej Transformaty Fouriera 12
3.4 Przetwarzenie kodu programu komputerowego na kod wykonywalny 12
3.5 Opis kompilacji programow w jezyku C 12
3.6 Poziomy i flagi optymalizacyjne kompilatora GCC 12
3.7 Zaawansowane techniki optymalizacji kodu 12

4 Techniki optymalizacji programów komputerowych **13**
4.1 Opis procesu kompilacji programu komputerowego 13
4.2 Programowe techniki optymalizacji programów komputerowych 13
4.3 Przegląd najpopularniejszych kompilatorów języka C 13
4.4 Poziomy optymalizacji dostępne w kompilatorach Clang i Gcc 13
4.5 Weryfikacja poprawności obliczeń z biblioteka zewnetrzna 13

5 Projekt systemu testowego i metodyka przeprowadzonych badań **14**
5.1 Założenia projektowe i architektura aplikacji 14
5.2 Opis plattformy sprzetowej 14
5.3 Użyte narzędzia programistyczne 14
5.4 Metodyka przeprowadzonych pomiarów 14
5.5 Prezentacja przygotowyanych wariantów programu 14
5.6 Implementacja wersji wielowątkowej, opis biblioteki pthread 14

5.7 Weryfikacja poprawności obliczeń z biblioteka zewnetrzna 14

6 Analiza wyników przeprowadzonych badan 15

6.1 Analiza wpływu flag kompilatora na kod sekwencyjny 15

6.2 Analiza wpływu optymalizacji ręcznej programu sekwencyjnego 15

6.3 Analiza skalowalności rozwiązania wielowątkowego 15

6.4 Zestawienie zbiorcze i dyskusja wyników 15

7 Podsumowanie i wnioski 16

7.1 Wnioski koncowe 16

7.2 kierunek dalszych badan 16

Spis rysunków 17

Spis tabel 18

LISTA SYMBOLI

LISTA SKRÓTÓW

1. WSTĘP I CEL PRACY

1.1. Cel pracy

1.2. Zawartość rozdziałów

Ogólną strukturę niniejszej pracy można przedstawić w następujący sposób:

- **Rozdział 2: Charakterystyka systemów wbudowanych opartych na systemie Linux.**
  Obejmuje charakterystykę systemów wbudowanych opartych na systemie Linux, z opisem podstawowych pojęc jak jadro systemu, menadzer procesow, zarządzanie pamięcią oraz mechanizmy wielozadaniowości. Opisuje specyfikę zarządzania zasobami sprzętowymi przez jądro systemu. Szczególną uwagę poświęcono mechanizmom wielozadaniowości, szeregowania wątków oraz zarządzania pamięcią wirtualną, które mają kluczowy wpływ na wydajność aplikacji.

- **Rozdział 3: Wybrany algorytm przetwarzania danych i techniki optymalizacji kodu.**
  Przedstawia teoretyczne podstawy algorytmu Szybkiej Transformaty Fouriera (FFT) oraz analizę jego złożoności obliczeniowej. Prezentuje różne implementacje tego algorytmu Omawia proces kompilacji kodu w języku C, nakreslając istotę wyboru kompilatora. Ponadto przedstawia poziomy optymalizacji oferowane przez kompilator GCC, używany w pracy. Wyjaśnia zaawansowane techniki programistyczne, takie jak instrukcje wektorowe oraz optymalizacja pod kątem dostępu dopamięci podręcznej.

- **Rozdział 4: Projekt i implementacja aplikacji testowej.**
  Opisuje szczegółowo opracowane oprogramowanie realizujące algorytm Szybkiej Transformaty Fouriera (FFT). Przedstawia alternatywne warianty implementacji kodu: wersję sekwencyjną , wersję zoptymalizowaną pod kątem dostępu do pamięci oraz wersję wielowątkową wykorzystującą mechanizmy współbieżności.

- **Rozdział 5: Środowisko badawcze i metodyka pomiarów.**
  Charakteryzuje wykorzystaną platformę sprzętową oraz konfigurację systemu operacyjnego Linux. Opisuje narzędzia programistyczne, biblioteki oraz frameworki użyte do implementacji i optymalizacji kodu. Definiuje scenariusze testowe, przygotowane konfiguracje procesu kompilacji (zestawy flag optymalizacyjnych) oraz narzędzia i metody użyte do pomiaru czasu wykonania i zużycia zasobów systemowych.

- **Rozdział 6: Analiza wpływu optymalizacji na wydajność systemu.**
  Prezentuje wyniki przeprowadzonych badań. Każdy podrozdział zawiera zestawienie i analizę wpływu konkretnych czynników – poziomu optymalizacji kompilatora, zmian w kodzie źródłowym oraz liczby wątków – na ostateczną wydajność aplikacji w systemie wbudowanym.

- **Rozdział 7: Podsumowanie i wnioski.**
  Stanowi syntezę uzyskanych wyników oraz ocenę skuteczności badanych metod optymalizacji. Zawiera wnioski końcowe dotyczące relacji między nakładem pracy programisty a automatyczną optymalizacją kompilatora oraz propozycje dalszych kierunków badań.

2. ARCHITEKTURA SYSTEMÓW OPERACYJNYCH W ZASTOSOWANIACH WBUDOWANYCH

Współczesna inżynieria systemów wbudowanych znajduje się w punkcie zwrotnym, w którym tradycyjne podejście oparte na bezpośrednim sterowaniu sprzętem (Bare-Metal) ustępuje miejsca zaawansowanym platformom zintegrowanym z systemami operacyjnymi klasy Linux. Ta ewolucja jest podyktowana rosnącą złożonością realizowanych zadań, wymagających nie tylko wysokiej mocy obliczeniowej, ale również sprawnego zarządzania współbieżnością, pamięcią wirtualną oraz interfejsami komunikacyjnymi.

System operacyjny, zgodnie z klasyczną definicją A. Tanenbauma, pełni w tym układzie rolę zarządcy zasobów, którego zadaniem jest sprawiedliwy i bezpieczny arbitraż dostępu do fizycznych komponentów platformy. Jednakże, w przypadku algorytmów o wysokiej intensywności obliczeniowej, takich jak Szybka Transformata Fouriera (FFT), ta warstwa abstrakcji staje się źródłem specyficznych wyzwań inżynierskich. Mechanizmy jądra, takie jak szeregowanie zadań czy obsługa przerwń, wprowadzają do systemu niedeterminizm, objawiający się fluktuacjami czasu wykonania kodu (jitterem) oraz narzutem administracyjnym (overhead).

Zrozumienie architektury systemu operacyjnego w zastosowaniach wbudowanych jest zatem kluczowe dla procesu optymalizacji. Niniejszy rozdział stanowi fundament teoretyczny dla dalszych badań, analizując drogę danych od fizycznego sygnału, przez mechanizmy izolacji przestrzeni jądra i użytkownika, aż po polityki planisty systemowego. Celem poniższych rozważań jest wskazanie wąskich gardeł systemowych, które mogą ograniczać teoretyczną wydajność algorytmów cyfrowego przetwarzania sygnałów na platformach o ograniczonych zasobach [1]

2.1. Rola systemów operacyjnych w systemach wbudowanych

Systemem wbudowanym możemy nazwać taki system komputerowy, którego rola jest ściśle określona. Cecha ta jest wyrózniajaca je od systemów komputerowych ogólnego przeznacznia. Wąska specjalizacja takich systemów pozwala na zagwarantowanie poprawnosci systemu.

2.1.1. Model warstwowy systemów operacyjnych

W inżynieri oprogramowania istotną rolę odgrywa koncepcja **abstrakcji**, polegająca na celowym ukryciu szczegółów implementacji danej funkcjonalności oprogramowania, przed jej potencjalnym użytkownikiem [1]. Podejście to opiera się na założeniu, że należy dostarczyć gotowe funkcjonalność wraz z odpowiednią dokumentacją opisującą sposób jej wykorzystania, zamiast wymagać od użytkownika zrozumienia wewnętrznych mechanizmów działania wytworzonego modułu. Taka praktyka w sposób znaczący podnosi wygodę wytwarzania nowych elementów systemu, umożliwia zrównoleglenie wytwarzania oprogramowania oraz pozwala na większą specjalizacje jej producentów. W celu realizacji komunikacji między kolejnymi modułami systemu stosuj się takie narzędzia jak **interfejsy programistyczne** (ang. *Application Programming Interface*, skrót **API**), czyli zbiór ściśle określonych reguł i sposobów komunikacji, które umożliwiają komunikację między różnymi warstwami oprogramowania. Narzędzie to jest niezbędne, aby w sposób przewidywalny i bezpieczny przekazywać danę lub sterowanie, między różnymi elemenetami systemu, bez konieczności znajmości szczegółów implementacji elementu systemy, który jest odbiorcą API.

Użycie abstrakcji jest również standardem w projektowaniu systemów operacyjnych. Wy-

nika to z wielu przyczyn, z których bardzo istotne to:

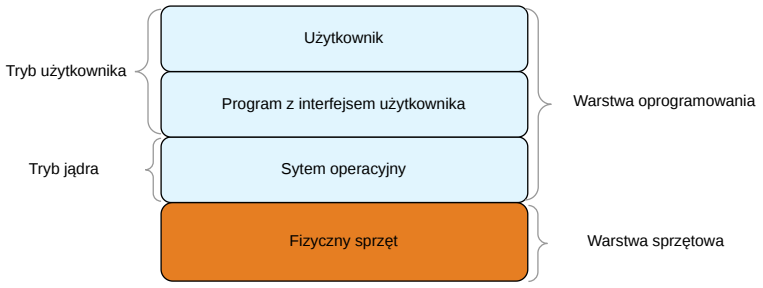
1. **Konieczność zapewnienia bezpieczeństwa systemu** - Systemy operacyjnego są w rzeczywistości rozwiązaniami o niezwykłej złożoności, których kod źródłowy sięga kilkudziesięciu milionów linii [1]. Użycie abstrakcji, w tym wypadku, wprowadza wyraźną hierarchię uprawnień, która oddziela ogólne operacje użytkownika od krytycznych zadań całego systemu. Dzięki temu mechanizmowi, błąd powstały w jednej części oprogramowania pozostaje odizolowany i nie wpływa na poprawne funkcjonowanie pozostałych elementów systemu, zarówno sprzętowych jak i programowych.

2. **Zapewnienie przenoszalności oprogramowania** - System wbudowany, jak każdy system komputerowy, składa się z fizycznego sprzętu i oprogramowania, które jest na nim uruchamiane. Te dwa elementy działają wspólnie, jednakże dąży się do tego, aby nie były one od siebie w pełni zależne, jeśli chodzi o swoją architekturę. Dzięki zastosowaniu pewnych abstrakcji, możliwe jest tworzenie oprogramowania, które będzie możliwe do uruchomienia na różnym sprzęcie fizycznym. Rozwiązanie to jest niezwykle korzystne, ponieważ pozwala na niezależny rozwój systemu zarówno pod kątem sprzętowym, jak i programowym.

3. **Umożliwienie korzystania z języków i narzędzi wysokiego poziomu** - Ukrycie technicznych detali działania sprzętu pozwala na wytwarzanie oprogramowania w językach wysokiego poziomu, w których większy nacisk kładzię się na tworzenie programów, które są spójne ze sposobem myślenia człowieka, zamiast skupiać się na bezpośredniej obsłudze fizycznych komponentów urządzenia elektornicznego, które wykonuje program.

Pojęcię abstrakcji jest kluczowe, aby zrozumieć architekture systemów operacyjnych. W literaturze spotyka się różne podziału na warstwy systemu komputerowego [1, 2]. Cechą wspólną dla propozycji podziału systemów komputerowych jest to, że wyodrębnia się w nich zasoby sprzętowe od programów na nich wykonywanych, tworząc odpowiednio **warstwę sprzętową i warstwę oprogramowania**. Do warstwy sprzętowej zaliczamy wszystkie urządzenia fizyczne, występujące w systemie takie jak na przykład **jeednostka centralna** (ang. **Central Processing Unit**, **CPU**), **pamięć operacyjna** (ang. **Random Access Memory**, **RAM**), oraz **urządzenia wejścia-wyjścia** (ang. **Input/Output Devices**, **I/O**). Do warstwy programowej zaliczamy zestaw programów aplikacyjnych, które definiują sposoby wykorzystania zasobów sprzętowych systemu w celu rozwiązywania konkretnych problemów użytkownika [2].

Na potrzebę niniejszej pracy przytoczona zostanie propozycja podziału systemu komputerowego zaprezentowana w książce [2], zaprezentowana na rysunku 2.1.



Rysunek 2.1: Warstwowy model systemu komputerowego

2.1.2. Systemy wbudowane czasu rzeczywistego wraz z ich podziałem

2.1.3. Z czego wynika koniecznosc stosowania systemow operacyjnych a nie bare metal

2.2. Budowa i architektura systemu Linux

2.2.1. Jądro systemu

2.2.2. Mechanizm stronnicowania pamięci i wykorzystanie pamieci cache

2.2.3. wykorzystanie systemu Linux w systemach wbudowanych

2.3. Procesy i wątki w systemie Linux

2.4. Szeregowani zadań i determinizm czasowy w systemach wbudowanych opartych na Linuxie

3. WYBRANY ALGORYTM PRZETWARZANIA DANYCH I TECHNIKI OPTYMALIZACJI KODU

3.1. Teorytyczne podstawy analizy algorytmów

3.2. Opis i analiza Szybkiej Transformaty Fouriera

3.3. Warianty algorytmu Szybkiej Transformaty Fouriera

3.4. Przetwarzenie kodu programu komputerowego na kod wykonywalny

3.5. Opis kompilacji programow w jezyku C

3.6. Poziomy i flagi optymalizacyjne kompilatora GCC

3.7. Zaawansowane techniki optymalizacji kodu

4. TECHNIKI OPTYMALIZACJI PROGRAMÓW KOMPUTEROWYCH

4.1. Opis procesu kompilacji programu komputerowego

4.2. Programowe techniki optymalizacji programów komputerowych

4.3. Przegląd najpopularniejszych kompilatorów języka C

4.4. Poziomy optymalizacji dostępne w kompilatorach Clang i Gcc

4.5. Weryfikacja poprawności obliczeń z biblioteka zewnetrzna

5. PROJEKT SYSTEMU TESTOWEGO I METODYKA PRZEPROWADZONYCH BADAŃ

5.1. Założenia projektowe i architektura aplikacji

5.2. Opis plattformy sprzetowej

5.3. Użyte narzędzia programistyczne

5.4. Metodyka przeprowadzonych pomiarów

5.5. Prezentacja przygotowyanych wariantów programu

5.6. Implementacja wersji wielowątkowej, opis biblioteki pthread

5.7. Weryfikacja poprawności obliczeń z biblioteka zewnetrzna

6. ANALIZA WYNIKÓW PRZEPROWADZONYCH BADAN

6.1. Analiza wpływu flag kompilatora na kod sekwencyjny

6.2. Analiza wpływu optymalizacji ręcznej programu sekwencyjnego

6.3. Analiza skalowalności rozwiązania wielowątkowego

6.4. Zestawienie zbiorcze i dyskusja wyników

7. PODSUMOWANIE I WNIOSKI

7.1. Wnioski koncowe

7.2. kierunek dalszych badan

SPIS RYSUNKÓW

2.1 Warstwowy model systemu komputerowego 10

SPIS TABEL

BIBLIOGRAFIA

[1] Andrew S. Tanenbaum and Herbert Bos. *Systemy operacyjne*. Helion, Gliwice, 2024. Przekład: Radosław Meryk.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. J. Wiley & Sons, Hoboken, N.J., 8th ed. edition, 2008.